

RMA TRACKER DOCUMENTATION LIBRARY

VERSION 2.1.1

VOLUME II

Technical Design & Architecture Manual



RMA Tracker

Provides a comprehensive technical reference for the architecture, design, implementation, database, user interface, packaging, and testing of RMA Tracker
Version 2.1.1.

Prepared for: Harp Enterprises

Prepared by: Brennen Gabriel

Document version: 2.1.1

Publication date: July 2026

Table of Contents

1.	<i>Introduction.....</i>	<i>8</i>
1.1.	Purpose.....	8
1.2.	Scope.....	8
1.3.	Intended Audience.....	9
1.4.	Relationship to the Documentation Library.....	10
1.5.	Document Conventions	11
1.6.	Definitions, Acronyms, and Abbreviations.....	13
1.7.	References	14
2.	<i>Architecture Overview</i>	<i>16</i>
2.1.	Architecture Summary.....	16
2.2.	Architectural Goals.....	16
2.3.	High-Level System Context	17
2.4.	Logical Architecture.....	17
2.5.	Physical Architecture.....	18
2.6.	Component Interaction	18
2.7.	Data Flow Overview	19
2.8.	Architectural Constraints.....	19
2.9.	Design Principles	19
2.10.	Architecture Diagrams (To be finished later).....	20
3.	<i>Section 3 – Project Structure.....</i>	<i>21</i>
3.1.	Repository Overview	21
3.2.	Source Directory Layout	21
3.3.	Package Structure	21
3.4.	Resource Structure	22
3.5.	Build Files	22
3.6.	Runtime Data Directories.....	22
3.7.	Naming Conventions	22
3.8.	Dependency Organization.....	22
3.9.	Configuration Management.....	23
3.10.	Project Structure Diagram.....	23

4.	<i>Technology Stack and Dependencies</i>	<i>24</i>
4.1.	Java Platform	24
4.2.	Java Swing.....	24
4.3.	SQLite	24
4.4.	Gradle.....	24
4.5.	JDBC	24
4.6.	PDF Generation Libraries.....	24
4.7.	Excel Export Libraries	25
4.8.	Logging and Diagnostics	25
4.9.	Packaging Tools.....	25
4.10.	Dependency Inventory.....	25
5.	<i>Technology Stack and Dependencies</i>	<i>26</i>
5.1.	Database Overview.....	26
5.2.	Data Model.....	26
5.3.	Entity Relationship Diagram	26
5.4.	Table Definitions.....	26
5.5.	Column Definitions.....	28
5.6.	Primary Keys	28
5.7.	Foreign Keys	28
5.8.	Indexes.....	29
5.9.	Constraints and Validation	29
5.10.	Database Initialization.....	29
5.11.	Schema Migration and Versioning.....	29
5.12.	CRUD Operations	29
5.13.	Transaction Handling.....	29
5.14.	Data Retention and Deletion.....	29
5.15.	Database Error Handling	29
6.	<i>Class and Package Documentation</i>	<i>30</i>
6.1.	Documentation Approach	30
6.2.	Package Overview.....	30
6.3.	Package Dependency Structure	31

6.4.	Application Entry Point.....	32
6.5.	Model Classes.....	32
6.6.	Data Access Classes.....	46
6.7.	Service Classes	48
6.8.	Reporting Classes.....	52
6.9.	User Interface Classes.....	53
6.10.	Utility Classes	67
6.11.	Exception and Error Propagation	67
6.12.	Class Relationships.....	68
6.13.	Method-Level Reference Guidance	68
6.14.	Extension Points.....	69
7.	<i>User Interface Design</i>	70
7.1.	UI Design Overview	70
7.2.	Main Dashboard	70
7.3.	Create RMA Window	70
7.4.	Edit RMA Window	70
7.5.	Search and Filtering.....	70
7.6.	Repair Item Management.....	70
7.7.	Shipping Interface.....	71
7.8.	Status History Interface	71
7.9.	Report Preview	71
7.10.	Backup and Restore Windows.....	71
7.11.	Settings Window	71
7.12.	Menus and Keyboard Shortcuts.....	71
7.13.	UI Validation and Error Messages.....	71
7.14.	Accessibility and Usability.....	71
7.15.	UI Component Diagrams.....	71
8.	<i>Reporting System</i>	73
8.1.	Reporting Architecture	73
8.2.	Report Data Sources.....	73
8.3.	Report Templates	73

8.4.	Report Preview	73
8.5.	PDF Export.....	73
8.6.	Excel Export	73
8.7.	Printing.....	73
8.8.	File Naming Conventions.....	73
8.9.	Reporting Error Handling	74
8.10.	Future Reporting Enhancements	74
9.	<i>Backup and Recovery System</i>	<i>75</i>
9.1.	Backup Architecture	75
9.2.	Automatic Backups.....	75
9.3.	Manual Backups.....	75
9.4.	Backup File Locations.....	75
9.5.	Backup Naming and Rotation	75
9.6.	Restore Process	75
9.7.	Validation and Safety Checks	75
9.8.	Recovery from Failure.....	75
9.9.	Backup Logging.....	76
9.10.	Backup Security.....	76
9.11.	Future Cloud Backup	76
10.	<i>Packaging and Distribution.....</i>	<i>77</i>
10.1.	Packaging Overview.....	77
10.2.	Gradle Build Process.....	77
10.3.	Executable JAR.....	77
10.4.	jpackage Configuration.....	77
10.5.	macOS Application Bundle	77
10.6.	macOS DMG	78
10.7.	Windows Packaging	78
10.8.	Linux Packaging	78
10.9.	Application Versioning	78
10.10.	Distribution Structure.....	78
10.11.	Code Signing and Notarization	78

10.12.	Packaging Troubleshooting	78
11.	<i>Testing Strategy and Quality Assurance</i>	<i>80</i>
11.1.	Testing Overview	80
11.2.	Unit Testing.....	80
11.3.	Integration Testing.....	80
11.4.	Database Testing.....	80
11.5.	User Interface Testing.....	80
11.6.	Reporting Tests	80
11.7.	Backup and Restore Tests.....	80
11.8.	Packaging Tests	80
11.9.	Regression Testing.....	81
11.10.	Manual Test Procedures.....	81
11.11.	Test Data	81
11.12.	Defect Tracking	81
11.13.	Release Readiness Criteria	81
11.14.	Test Results and Evidence	81
12.	<i>Security, Reliability, and Performance Design</i>	<i>82</i>
12.1.	Current Security Model.....	82
12.2.	Data Protection.....	82
12.3.	Input Validation	82
12.4.	Error Handling Strategy	82
12.5.	Reliability Design.....	82
12.6.	Performance Goals	82
12.7.	Database Performance.....	82
12.8.	UI Responsiveness.....	82
12.9.	Logging and Diagnostics.....	83
12.10.	Future Authentication and Authorization.....	83
13.	<i>. Maintenance and Future Architecture</i>	<i>84</i>
13.1.	Maintenance Overview	84
13.2.	Adding New Features.....	84
13.3.	Modifying the Database	84

13.4.	Adding New Reports	84
13.5.	Updating Dependencies.....	84
13.6.	Refactoring Guidance	84
13.7.	Planned Version 2.2 Architecture	85
13.8.	Planned Version 3.0 Architecture	85
13.9.	Cloud Synchronization.....	85
13.10.	User Accounts and Roles	85
13.11.	Automatic Tracking Updates	85
13.12.	API and Integration Layer.....	85
13.13.	Scalability Considerations	85
14.	<i>. Appendices</i>	<i>87</i>
14.1.	Package Diagrams	87
14.2.	Class Diagrams.....	87
14.3.	Database Schema	87
14.4.	Entity Relationship Diagram.....	87
14.5.	Sequence Diagrams.....	87
14.6.	Activity Diagrams	87
14.7.	Deployment Diagram	88
14.8.	Project Directory Tree	88
14.9.	Dependency Inventory.....	88
14.10.	Build and Packaging Commands	88
14.11.	Database Queries and Scripts.....	88
14.12.	Test Case Catalog.....	88
14.13.	Technical Glossary	88
14.14.	References.....	88
14.15.	Revision History.....	88

1. Introduction

This section introduces the Technical Design & Architecture Manual for RMA Tracker Version 2.1.1. It defines the purpose and scope of the manual, identifies its intended audience, explains its relationship to the other documents in the RMA Tracker documentation library, and establishes the conventions and references used throughout the volume. The section provides the context required to understand the technical descriptions, design decisions, diagrams, database documentation, class references, testing records, and maintenance guidance presented in later sections.

1.1. Purpose

The purpose of this manual is to provide a complete technical description of how RMA Tracker Version 2.1.1 is designed, organized, implemented, built, tested, packaged, and maintained. Whereas the Software Requirements Specification defines what the application is required to do, this volume explains how the implemented software fulfills those requirements through its architecture, source-code organization, database structure, user interface, reporting features, backup mechanisms, and deployment process.

This manual serves as the primary engineering reference for the current release. It records the technical structure of the application so that future developers and maintainers can understand the system without relying solely on source-code inspection or informal knowledge. It also preserves the reasoning behind important implementation decisions, including the use of a Java desktop architecture, an embedded SQLite database, Gradle-based builds, local file storage, report generation, and platform-specific packaging.

The manual is intended to support software maintenance, defect correction, regression testing, release preparation, technical review, onboarding of new contributors, and planning for future versions. It provides enough detail for a qualified developer to trace a feature from the user interface through the model, service, repository, and database layers, and to understand the supporting files and runtime resources involved.

This document is descriptive rather than prescriptive in areas where Version 2.1.1 has already been implemented. It documents the actual architecture and behavior of the release. Where future improvements are discussed, they are identified clearly as planned, proposed, or recommended so that the current system is not confused with future design goals.

1.2. Scope

This manual covers the technical design and implementation of the RMA Tracker Version 2.1.1 desktop application. Its scope includes the following areas:

- The overall application architecture and the responsibilities of the presentation, business logic, persistence, reporting, backup, and utility components.

- The project repository, source directory structure, Java packages, resource files, Gradle configuration, dependencies, runtime directories, and generated artifacts.
- The SQLite database, including entities, tables, columns, keys, relationships, initialization behavior, CRUD operations, transactions, integrity controls, and error handling.
- The application classes and interfaces, including model objects, repository classes, services, Swing windows and dialogs, report-preview components, status-history components, and shared utilities.
- The design of the user interface, including the main window, RMA creation and editing dialogs, search and filtering controls, repair-item management, shipping information, report preview, status history, backup, and restore workflows.
- The reporting subsystem, including on-screen previews, printing, PDF export, Excel export, data selection, formatting rules, and generated-file conventions.
- The backup and recovery subsystem, including automatic backups, manual backups, restore procedures, application-support directories, file naming, safety checks, and failure recovery.
- The build, packaging, and distribution process, including Gradle tasks, executable artifacts, jpackage configuration, macOS application bundles and disk images, and planned Windows packaging.
- The testing strategy for model behavior, persistence, large RMA records, reporting, backup and restore, packaging, and release readiness.
- Technical constraints, reliability considerations, security limitations, maintainability guidance, and future architectural directions.

This manual does not replace the end-user operating instructions or the deployment and maintenance procedures provided in other volumes. It also does not define business requirements independently of the approved Software Requirements Specification. Source-code excerpts may be included to explain important structures or algorithms, but the source repository remains the authoritative implementation record.

1.3. Intended Audience

The manual is written for readers who require a technical understanding of RMA Tracker. The expected audiences are described below.

Audience	Primary Use
Software developers	Developers use this manual to understand the application architecture, package responsibilities, class relationships, database access patterns, UI workflows,

	extension points, and build process before modifying or extending the software.
Software testers and quality-assurance reviewers	Testers use the documented design, data flows, validation rules, and component responsibilities to create test cases, reproduce defects, assess regression risk, and verify release readiness.
System administrators and technical support personnel	Administrators and support personnel use the sections on runtime directories, database storage, backups, restore operations, logging, packaging, and troubleshooting to diagnose and recover the application.
Future maintainers and contributors	Future contributors use the manual as an onboarding reference and as a technical history of the application. The document helps reduce dependency on undocumented institutional knowledge.
Technical reviewers and stakeholders	Reviewers may use the manual to evaluate the completeness, maintainability, reliability, and commercial readiness of the application design.
Project owner and documentation maintainers	The project owner uses this volume to keep architecture records synchronized with future releases and to identify sections that must be revised when the implementation changes.

Readers are expected to have basic familiarity with object-oriented programming, Java, relational databases, desktop user-interface concepts, and software testing. More specialized topics are explained where necessary, and key terms are defined in Section 1.6 and the technical glossary in the appendices.

1.4. Relationship to the Documentation Library

RMA Tracker Version 2.1.1 is documented through a coordinated four-volume library. Each volume has a distinct purpose, but the volumes should be read together when a complete understanding of the product is required.

Volume	Relationship to Volume II
--------	---------------------------

Volume I - Software Requirements Specification	Defines what the application must do, the functional and non-functional requirements, data requirements, business rules, use cases, future enhancements, and acceptance criteria. Volume II traces the implemented technical design back to these requirements.
Volume II - Technical Design & Architecture Manual	Explains how the application is built. It documents the architecture, project structure, technology stack, database, classes, user interface, reporting, backup, packaging, testing, and future technical direction.
Volume III - User Manual & Administration Guide	Explains how end users and administrators operate the application. It focuses on tasks such as creating and editing RMAs, managing repair items, searching, reporting, backing up, restoring, and troubleshooting routine usage.
Volume IV - Installation, Deployment & Maintenance Guide	Explains how to install, build, package, distribute, update, recover, and maintain the application over time. It uses the architectural and packaging information recorded in Volume II as its technical foundation.

When a technical topic is governed by an approved requirement, the relevant requirement identifier from Volume I should be referenced. When a design decision affects user procedures or deployment practices, the corresponding sections of Volumes III and IV should be cross-referenced. This approach keeps the documentation library consistent while avoiding unnecessary duplication.

1.5. Document Conventions

The following conventions are used throughout this manual to present technical information consistently.

Convention	Usage
Section numbering	Major sections use whole numbers. Subsections use decimal numbering, such as

	5.3 or 8.4.2. Numbering remains continuous throughout the single Volume II document.
Class and interface names	Java class, interface, enum, and package names are shown using their exact source-code spelling, for example <code>RmaRecord</code> , <code>BackupService</code> , <code>Status</code> , and <code>com.harpEnterprises.Brennen</code> .
Methods and fields	Method names include parentheses when discussed as executable operations, such as <code>save()</code> , <code>search()</code> , or <code>createBackup()</code> . Field and variable names are shown using their exact code spelling.
File and directory names	File names, folder names, extensions, paths, Gradle tasks, and command-line entries are shown in a monospace or code style when formatting permits.
Database objects	Table names, column names, keys, and SQL statements use the exact identifiers defined in the implemented schema.
Requirement references	Requirements from Volume I are referenced by their identifiers, such as FR-1, NFR-3, DR-2, or BR-4.
Figures and tables	Figures, diagrams, and tables are numbered by section and supplied with descriptive captions. Cross-references use the assigned figure or table number.
Status labels	Application status values are written using their user-facing labels when describing the interface and their enum constants when describing source code.
Version distinctions	Implemented behavior is described in the present tense. Proposed or future behavior is explicitly labeled as planned, proposed, recommended, or future.
Warnings and notes	Important operational risks, implementation limitations, or maintenance cautions are

	identified as notes, warnings, or technical considerations.
--	---

The manual uses the product name “RMA Tracker” and the release designation “Version 2.1.1.” The term “application” refers to the installed desktop software. The term “system” may refer to the complete combination of application code, database, generated files, backups, reports, and runtime environment.

1.6. Definitions, Acronyms, and Abbreviations

The following technical terms are used frequently throughout this manual. A more extensive technical glossary is provided in the appendices.

Term	Definition
API	Application Programming Interface; a defined mechanism through which software components or external systems communicate.
CRUD	Create, Read, Update, and Delete operations performed on persistent data.
DAO	Data Access Object; a class or component responsible for persistence operations. In this project, repository classes perform this role.
DBMS	Database Management System.
ERD	Entity Relationship Diagram; a diagram showing database entities and their relationships.
GUI	Graphical User Interface.
JAR	Java Archive; a packaged file containing compiled Java classes and resources.
JDBC	Java Database Connectivity; the Java API used to communicate with SQLite.
JDK	Java Development Kit.
JRE	Java Runtime Environment.
PDF	Portable Document Format.

RMA	Return Merchandise Authorization.
Repository	A persistence component that reads and writes domain objects to the database.
SQLite	The embedded relational database used by RMA Tracker.
Swing	The Java desktop user-interface toolkit used by RMA Tracker Version 2.1.1.
UML	Unified Modeling Language.
XLSX	The Microsoft Excel Open XML workbook format.

1.7. References

The following documents, standards, technologies, and implementation resources provide context for this manual. Exact library versions and licenses are documented in the dependency inventory later in Volume II.

Reference	Relevance
RMA Tracker v2.1.1 - Volume I: Software Requirements Specification	Primary source for approved functional requirements, non-functional requirements, data requirements, business rules, use cases, future enhancements, and acceptance criteria.
RMA Tracker v2.1.1 source repository	Authoritative implementation record for Java source code, tests, Gradle files, resources, and packaging configuration.
Java Platform documentation	Reference for the Java language, standard library, Swing toolkit, printing APIs, file handling, and runtime behavior.
SQLite documentation	Reference for embedded database behavior, SQL syntax, constraints, transactions, and database-file characteristics.
SQLite JDBC driver documentation	Reference for Java-to-SQLite connectivity and driver-specific behavior.

Gradle documentation	Reference for build automation, dependency management, testing tasks, and artifact generation.
JUnit 5 documentation	Reference for unit-testing structure, annotations, assertions, and Gradle test integration.
Apache POI documentation	Reference for Microsoft Excel workbook creation and XLSX export, where applicable.
PDF generation library documentation	Reference for PDF document creation, layout, export, and file output used by the implemented reporting subsystem.
Oracle jpackage documentation	Reference for creating native application images and platform-specific installers.
IEEE/ISO/IEC 42010	Reference framework for architecture descriptions and architectural viewpoints.
ISO/IEC/IEEE 29148	Reference for requirements engineering and traceability between requirements and design documentation.
Unified Modeling Language specification	Reference for class, package, sequence, activity, and deployment diagrams included in this manual.

References should be updated whenever dependencies, development tools, packaging procedures, standards, or related volumes change. Where a referenced technology has multiple releases, the version actually used by RMA Tracker Version 2.1.1 should be recorded in the Technology Stack and Dependency Inventory sections.

Section summary. *This introduction establishes Volume II as the authoritative technical record for RMA Tracker Version 2.1.1. The next section presents the architecture overview and explains how the application's major components work together.*

2. Architecture Overview

This section documents the overall software architecture of RMA Tracker Version 2.1.1. It explains how the application is organized internally, how the major software components communicate, and the architectural decisions that support maintainability, reliability, performance, and future expansion. Whereas Volume I defines what the system must accomplish, this manual explains how those requirements are implemented.

2.1. Architecture Summary

RMA Tracker Version 2.1.1 is a standalone desktop application developed in Java using the Swing GUI framework and an embedded SQLite database. The application follows a layered architecture that separates the user interface, business logic, persistence, reporting, backup, and utility services into independent components.

The presentation layer consists of windows, dialogs, menus, tables, and forms through which users manage Return Merchandise Authorization (RMA) records. Business services coordinate validation, workflow processing, report generation, exports, and backup operations. Repository classes encapsulate database access, ensuring SQL operations remain isolated from the user interface. Utility classes provide reusable functionality such as file handling, date formatting, configuration management, and helper methods.

This layered approach improves readability, simplifies maintenance, reduces coupling, and creates a foundation for future enhancements such as user authentication, cloud synchronization, dashboards, REST APIs, and multi-user support.

2.2. Architectural Goals

The architecture was designed around several engineering objectives.

Maintainability: Organize related functionality into logical packages and classes that are easy to understand and modify.

Modularity: Separate presentation, business logic, persistence, reporting, and backup functionality into distinct layers.

Reliability: Protect application data through validation, transactional database operations where appropriate, and automatic backup procedures.

Performance: Provide responsive searching, filtering, reporting, and export capabilities while minimizing memory usage.

Extensibility: Support future features without major redesign by maintaining loose coupling between components.

Portability: Use Java and Gradle to simplify deployment across supported desktop operating systems.

Usability: Present a consistent interface that minimizes training requirements and reduces user error.

2.3. High-Level System Context

At a high level, the application consists of a desktop client, an embedded SQLite database, generated reports, exported documents, and local backup storage.

Typical workflow:

1. A user interacts with the Swing interface.
2. Business services validate requests.
3. Repository classes perform database operations.
4. SQLite stores or retrieves application data.
5. Updated information is returned to the interface.
6. Optional reporting, PDF export, Excel export, printing, or backup services process the same validated data.

No external servers are required for normal operation, making deployment straightforward while maintaining full functionality.

2.4. Logical Architecture

The logical architecture is divided into six primary layers.

Presentation Layer

Responsible for all windows, dialogs, menus, tables, forms, and user interaction.

Business Layer

Implements validation, workflow rules, report coordination, backup coordination, and application logic.

Persistence Layer

Contains repository classes responsible for CRUD operations against the SQLite database.

Reporting Layer

Generates printable reports, PDF documents, Excel exports, and print previews.

Backup & Recovery Layer

Creates automatic and manual backups, validates restore operations, and protects user data.

Utility Layer

Provides reusable helper functions including formatting, configuration, file management, and common utilities.

Each layer communicates only with adjacent layers, reducing dependencies and improving maintainability.

2.5. Physical Architecture

RMA Tracker is packaged as a standalone desktop application using jpackage. The application executes entirely on the client workstation.

Runtime components include:

- Executable application (.app or platform equivalent)
- Java Runtime
- Embedded SQLite database
- Configuration files
- Generated reports
- Exported PDF and Excel documents
- Automatic backup directory
- Manual backup directory
- Application resources such as icons and templates

Because all components execute locally, the system operates without network connectivity for standard business functions while maintaining fast response times and simplified deployment.

2.6. Component Interaction

The architecture is composed of cooperating components that communicate through well-defined responsibilities.

The Main Window serves as the primary entry point for user interaction. User actions such as creating an RMA, editing records, searching, filtering, generating reports, or initiating backups are forwarded to business services.

Business services coordinate validation, workflow execution, and repository operations. Repository classes encapsulate all SQL statements and provide a consistent persistence interface. This separation ensures that presentation classes never communicate directly with SQLite.

Reporting services retrieve validated data through repositories and generate report previews, printable documents, PDF exports, and Excel workbooks. Backup services operate independently of reporting but use repository and configuration information to protect application data.

Component interactions are intentionally unidirectional wherever possible to reduce coupling, simplify testing, and improve maintainability.

2.7. Data Flow Overview

Data follows a predictable lifecycle throughout the application.

- User enters or modifies information through the graphical interface.
- Input is validated for completeness and correctness.
- Business rules are applied before persistence.
- Repository classes execute database operations.
- SQLite stores or retrieves data.
- Updated records are returned to the user interface.
- Reports, exports, and print previews retrieve data through the same service layer.
- Automatic or manual backups preserve database contents for recovery.

Using a consistent flow prevents duplicate logic, ensures validation occurs before persistence, and simplifies troubleshooting.

2.8. Architectural Constraints

Several architectural constraints influenced the implementation.

The application is designed for local desktop execution rather than a distributed environment. SQLite provides embedded storage, eliminating the need for a database server while limiting concurrent multi-user access. The project targets the Java platform and is built using Gradle, allowing consistent builds across supported operating systems.

The current architecture assumes a single-user environment and local file storage. Authentication, role-based security, cloud synchronization, REST APIs, and centralized databases are intentionally deferred to future versions to preserve architectural simplicity in Version 2.1.1.

2.9. Design Principles

The architecture follows established software engineering principles.

Separation of Concerns separates presentation, business logic, persistence, reporting, and utilities.

Single Responsibility ensures that each class performs one primary function.

Encapsulation hides implementation details behind well-defined interfaces.

Modularity organizes related functionality into cohesive packages.

Reusability minimizes duplicated code by centralizing shared logic.

Defensive Programming validates input, handles exceptional conditions gracefully, and protects application data from corruption.

These principles collectively improve readability, maintainability, scalability, and long-term evolution of the application.

2.10. Architecture Diagrams (To be finished later)

The completed technical manual will include a collection of architectural diagrams that visually describe the implementation.

Planned diagrams include:

- High-Level System Context Diagram
- Layered Architecture Diagram
- Component Diagram
- Package Dependency Diagram
- Deployment Diagram
- Database Entity Relationship Diagram
- Data Flow Diagram
- Sequence Diagrams for major workflows
- Activity Diagrams for Create RMA, Search, Reporting, Backup, and Restore

Figure 2.10-1 High-Level System Context Diagram

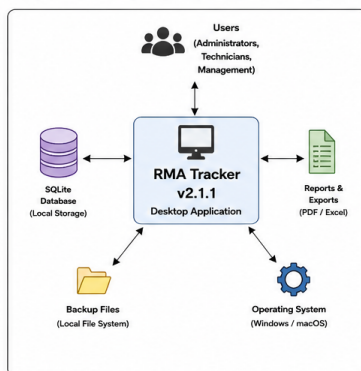


Figure 2.10-2 Logical Layered Architecture

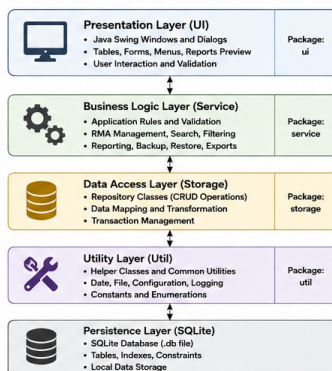


Figure 2.10-3 Physical Architecture / Deployment Diagram

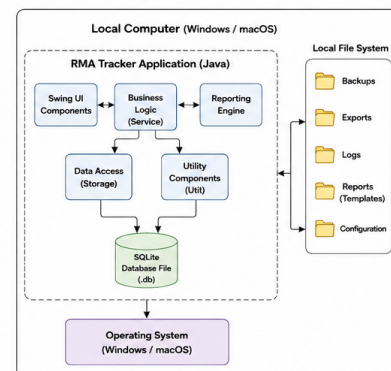


Figure 2.10-4 Component Interaction Diagram

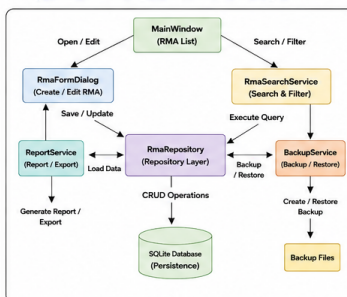


Figure 2.10-5 Data Flow Diagram (Create RMA Process)

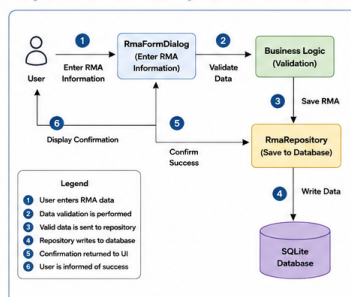
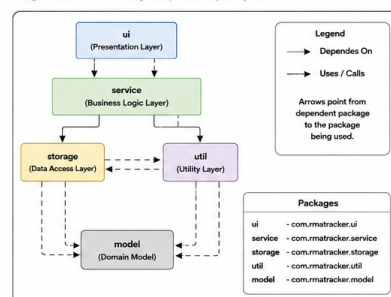


Figure 2.10-6 Package Dependency Diagram



3. Section 3 – Project Structure

Section 3 documents the internal organization of the RMA Tracker source code, project resources, build configuration, runtime directories, and naming conventions. It serves as the primary reference for developers who need to understand how the project is organized, built, and maintained. The project structure was designed to separate user interface components, business logic, persistence, reporting, and utility functions into cohesive packages that promote maintainability and future expansion.

3.1. Repository Overview

The RMA Tracker repository contains all source code, resources, build scripts, documentation, and supporting assets required to build and maintain the application. The repository is organized into logical top-level directories that separate production code, tests, documentation, generated artifacts, and build configuration.

3.2. Source Directory Layout

The application follows the standard Gradle project layout.

Typical directories include:

- src/main/java – Application source code
- src/main/resources – Icons and bundled resources
- src/test/java – Unit and integration tests
- build/ – Generated build artifacts
- docs/ – Documentation library
- gradle/ – Gradle wrapper files

This layout provides consistency with Java development best practices.

3.3. Package Structure

The application is organized into packages with clearly defined responsibilities.

ui

Contains windows, dialogs, menus, forms, tables, and presentation logic.

model

Defines domain objects including RMA records, repair items, shipping information, status history, and search criteria.

service

Implements business logic including searching, reporting, validation, and backup coordination.

storage

Encapsulates SQLite persistence through repository classes and the database manager.

util

Provides reusable helper classes such as date utilities, file handling, configuration, and formatting.

3.4. Resource Structure

Application resources include icons, report templates, images, configuration files, and other non-code assets bundled with the application. Resources are separated from source code to simplify packaging and maintenance.

3.5. Build Files

Gradle manages compilation, dependency resolution, testing, packaging, and distribution. Core project files include build.gradle, settings.gradle, the Gradle wrapper, and platform-specific packaging configuration.

3.6. Runtime Data Directories

During execution the application maintains several runtime directories.

- SQLite database location
- Automatic backup directory
- Manual backup directory
- Export directory
- Generated reports
- Application configuration

Separating runtime data from application binaries simplifies upgrades and protects user information.

3.7. Naming Conventions

Packages use lowercase names, classes use PascalCase, methods and variables use camelCase, constants use UPPER_SNAKE_CASE, and database tables and columns follow consistent descriptive naming. Version numbers follow semantic versioning.

3.8. Dependency Organization

External libraries are managed through Gradle while internal dependencies are limited to clearly defined package relationships. The presentation layer depends on services, services depend on storage and utilities, and repositories communicate with SQLite.

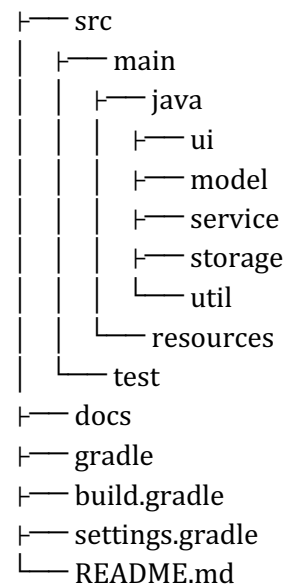
3.9. Configuration Management

Version-controlled configuration files define build behavior and application settings. Runtime configuration is isolated from source code to simplify deployment and maintenance while preserving portability across operating systems.

3.10. Project Structure Diagram

Recommended directory tree:

RMA Tracker



A graphical package hierarchy diagram will be added in the completed manual.

4. Technology Stack and Dependencies

This section documents the technologies, frameworks, libraries, build tools, and external dependencies used to develop, package, and maintain RMA Tracker Version 2.1.1. Each technology was selected to support a lightweight, maintainable desktop application while providing a solid foundation for future enhancements.

4.1. Java Platform

RMA Tracker is developed using Java, providing object-oriented design, platform independence, strong library support, and long-term maintainability. Java serves as the core runtime environment for all application logic, user interface processing, reporting, and data management.

4.2. Java Swing

Version 2.1.1 uses Java Swing as the graphical user interface framework. Swing provides windows, dialogs, menus, tables, forms, buttons, and event handling while remaining fully integrated with the Java platform. The interface follows a traditional desktop application model suitable for business users.

4.3. SQLite

SQLite is used as the embedded relational database. It requires no separate database server, stores all application data in a single database file, supports SQL transactions, and simplifies deployment for single-user desktop environments.

4.4. Gradle

Gradle manages compilation, dependency resolution, automated builds, testing, and packaging. The Gradle Wrapper ensures consistent build behavior across developer workstations without requiring a globally installed Gradle version.

4.5. JDBC

Java Database Connectivity (JDBC) provides the interface between the application and SQLite. Repository classes execute SQL statements through JDBC while insulating the remainder of the application from database implementation details.

4.6. PDF Generation Libraries

PDF export functionality uses a dedicated Java PDF library to generate printable reports while preserving formatting, layout, and portability. The reporting subsystem creates documents suitable for distribution and archival.

4.7. Excel Export Libraries

Excel export functionality uses Apache POI to generate XLSX workbooks. Exported spreadsheets preserve structured RMA data, making it easy to analyze, archive, or share information outside the application.

4.8. Logging and Diagnostics

The application uses exception handling, console diagnostics, and structured error messages to assist with troubleshooting. Future versions may introduce a dedicated logging framework for configurable logging levels and persistent log files.

4.9. Packaging Tools

Application packaging is performed with jpackage, producing native desktop distributions such as macOS application bundles and DMG installers. Future versions may include Windows installers and Linux packages.

4.10. Dependency Inventory

Core technologies include:

- Java Runtime
- Java Swing
- SQLite
- JDBC Driver
- Gradle
- Apache POI
- PDF generation library
- JUnit 5
- jpackage

Each dependency is selected to support portability, maintainability, and efficient desktop deployment.

5. Technology Stack and Dependencies

This chapter documents the actual SQLite database used by RMA Tracker Version 2.1.1. It describes the database architecture, schema, tables, relationships, persistence strategy, and integrity rules implemented by the application.

5.1. Database Overview

RMA Tracker stores operational data in an embedded SQLite database. The database resides locally with the application, eliminating the need for a database server while supporting ACID transactions, portability, and simplified deployment.

5.2. Data Model

The schema is centered on the RMA record. Repair items, shipping information, and status history reference the parent RMA through the `rma_id` foreign key, creating one-to-many relationships.

5.3. Entity Relationship Diagram

Relationships: RMAs (1) -> Repair Items (Many); RMAs (1) -> Shipping Information (Many); RMAs (1) -> Status History (Many). A graphical ERD should be inserted here in the final manual.

5.4. Table Definitions

The application currently contains four business tables: `rmAs`, `repair_items`, `status_history`, `shipping_information`.

Table: `rmAs`

Column	Type	Not Null	Default	PK	Description
<code>id</code>	INTEGER	No		Yes	
<code>rma_number</code>	TEXT	Yes		No	
<code>date_sent</code>	TEXT	No		No	
<code>date_received</code>	TEXT	No		No	

status	TEXT	Yes		No	
outgoing_tracking_number	TEXT	No		No	
return_tracking_number	TEXT	No		No	
notes	TEXT	No		No	
created_at	TEXT	Yes	CURRENT_TIMESTAMP	No	
updated_at	TEXT	Yes	CURRENT_TIMESTAMP	No	

Table: repair_items

Column	Type	Not Null	Default	PK	Description
id	INTEGER	No		Yes	
rma_id	INTEGER	Yes		No	
county	TEXT	Yes		No	
machine_type	TEXT	Yes		No	
serial_number	TEXT	No		No	
problem_description	TEXT	No		No	
repair_description	TEXT	No		No	
version	TEXT	No		No	
received	INTEGER	Yes	0	No	

Table: status_history

Column	Type	Not Null	Default	PK	Description
id	INTEGER	No		Yes	
rma_id	INTEGER	Yes		No	
old_status	TEXT	No		No	
new_status	TEXT	Yes		No	
changed_at	TEXT	Yes		No	

Table: shipping_information

Column	Type	Not Null	Default	PK	Description
id	INTEGER	No		Yes	
rma_id	INTEGER	Yes		No	
carrier	TEXT	Yes		No	
tracking_number	TEXT	Yes		No	
shipping_direction	TEXT	Yes		No	
created_at	TEXT	Yes	CURRENT_TIMESTAMP	No	

5.5. Column Definitions

The tables above document the implemented schema. Business-required fields are marked NOT NULL. Timestamp fields use CURRENT_TIMESTAMP defaults where applicable.

5.6. Primary Keys

Each business table uses an INTEGER primary key named id.

5.7. Foreign Keys

The rma_id column links repair_items, shipping_information, and status_history to the parent RMA record.

5.8. Indexes

Future versions should add indexes on `rma_number`, `status`, `date_sent`, `date_received`, and `tracking_number` to improve search performance as the database grows.

5.9. Constraints and Validation

Required fields are enforced through NOT NULL constraints and additional validation within the application.

5.10. Database Initialization

DatabaseManager creates the database and schema automatically if they do not exist.

5.11. Schema Migration and Versioning

Future schema changes should be managed through versioned migration scripts to preserve user data.

5.12. CRUD Operations

Repository classes implement Create, Read, Update, and Delete operations for each business entity while isolating SQL from the UI.

5.13. Transaction Handling

Related database operations should execute within transactions to ensure consistency and support rollback on failure.

5.14. Data Retention and Deletion

Business records remain in the database until intentionally removed. Future versions may introduce archival capabilities.

5.15. Database Error Handling

Database exceptions are handled by repository classes and surfaced to users through clear error messages while protecting data integrity.

6. Class and Package Documentation

This section documents the implemented Java packages and principal classes that compose RMA Tracker Version 2.1.1. It is based on the production source tree contained in the RMA Tracker project and identifies each class's responsibility, architectural role, important relationships, and public interface. The chapter is intended to help developers, reviewers, testers, and future maintainers understand where functionality resides and how the codebase should be extended without weakening separation of concerns.

6.1. Documentation Approach

Class documentation is organized first by package and then by individual type. Each class entry identifies its type, responsibility, inheritance or interface relationship, important fields, constructors, and public or protected operations detected in the implementation. Private implementation details are summarized when they materially affect architectural behavior, but the chapter does not reproduce the source code line by line.

The documentation uses the following classification:

- Domain model - represents persistent or workflow data.
- Repository - encapsulates database access and aggregate persistence.
- Service - performs business, export, backup, or search operations.
- User-interface component - presents data or collects user input through Swing.
- Report component - constructs printable or previewable report content.
- Utility - provides reusable cross-cutting helper behavior.
- Entry point - starts and initializes the application.

6.2. Package Overview

Package	Primary Responsibility	Production Types
com.harpenrprises.rmatracker	Root package containing the application entry point.	1
com.harpenrprises.rmatracker.model	Domain models and enumerations representing RMA business data.	9
com.harpenrprises.rmatracker.report	Report-construction components shared by	1

Package	Primary Responsibility	Production Types
	preview and print workflows.	
com.harpenrprises.rmatracker.service	Business and infrastructure services for search, export, and backup operations.	4
com.harpenrprises.rmatracker.storage	SQLite connection, schema, and repository-based persistence components.	3
com.harpenrprises.rmatracker.ui	Swing windows, dialogs, panels, menus, and action components.	14
com.harpenrprises.rmatracker.util	Reusable, application-wide helper utilities.	1

6.3. Package Dependency Structure

The package dependency direction generally flows from the Swing user interface toward services and repositories, while model and utility packages remain broadly reusable. The intended dependency sequence is:

ui -> service/report -> storage -> SQLite

The model package is shared by the UI, service, report, and storage packages. The util package provides cross-cutting support without owning business workflows. This arrangement reduces direct database coupling in presentation code and allows report, search, and backup behavior to evolve independently.

6.4. Application Entry Point

Root package containing the application entry point.

Type	Kind	Responsibility
Main	Class	Application entry point that initializes the desktop user interface on the Swing event-dispatch thread.

Main

Application entry point that initializes the desktop user interface on the Swing event-dispatch thread.

Package	com.harpenrprises.rmatracker
Type	Class
Source file	com/harpenrprises/rmatracker/Main.java
Extends	None
Implements	None

Public and protected operations

Visibility	Return Type	Method	Parameters
public static	void	main	String[] args

6.5. Model Classes

Domain models and enumerations representing RMA business data.

Type	Kind	Responsibility
Customer	Class	Represents customer or organizational contact information associated with an RMA workflow.
RepairItem	Class	Domain model for an individual machine or

Type	Kind	Responsibility
		device associated with an RMA.
RepairItemSearchResult	Class	Read-only projection used to display repair-item search matches with parent RMA context.
RmaRecord	Class	Primary aggregate model representing an RMA and its dates, status, notes, repair items, and shipping records.
RmaSearchCriteria	Class	Encapsulates optional search and filtering values used to query RMA data.
ShippingDirection	Enum	Enumeration distinguishing outbound and return shipment records.
ShippingInfo	Class	Domain model for carrier, tracking number, and shipment direction information.
Status	Enum	Enumeration of supported RMA lifecycle states.
StatusHistory	Class	Represents a chronological status transition recorded for an RMA.

Customer

Represents customer or organizational contact information associated with an RMA workflow.

Package	com.harpenterprises.rmatracker.model
Type	Class
Source file	com/harpenterprises/rmatracker/model/Customer.java
Extends	None
Implements	None

Key fields

Visibility	Static	Type	Name
private	No	String	companyName
private	No	String	contactName
private	No	String	phone
private	No	String	email
private	No	String	address
private	No	String	city
private	No	String	state
private	No	String	zip

Constructors

- public Customer()
- public Customer(String companyName, String contactName, String phone, String email, String address, String city, String state, String zip)

Public and protected operations

Visibility	Return Type	Method	Parameters
public	String	getCompanyName	None
public	void	setCompanyName	String companyName
public	String	getContactName	None
public	void	setContactName	String contactName
public	String	getPhone	None
public	void	setPhone	String phone
public	String	getEmail	None
public	void	setEmail	String email
public	String	getAddress	None
public	void	setAddress	String address
public	String	getCity	None
public	void	setCity	String city
public	String	getState	None
public	void	setState	String state
public	String	getZip	None
public	void	setZip	String zip
public	String	toString	None

RepairItem

Domain model for an individual machine or device associated with an RMA.

Package	com.harpenenterprises.rmatracker.model
Type	Class
Source file	com/harpenenterprises/rmatracker/model/RepairItem.java
Extends	None
Implements	None

Key fields

Visibility	Static	Type	Name
private	No	String	county
private	No	String	machineType
private	No	String	serialNumber
private	No	String	version
private	No	String	problemDescription
private	No	String	repairDescription
private	No	boolean	received

Constructors

- public RepairItem()
- public RepairItem(String county, String machineType, String serialNumber, String version, String problemDescription, String repairDescription)
- public RepairItem(String county, String machineType, String serialNumber, String version, String problemDescription, String repairDescription, boolean received)

Public and protected operations

Visibility	Return Type	Method	Parameters
public	String	getCounty	None
public	void	setCounty	String county
public	String	getMachineType	None
public	void	setMachineType	String machineType
public	String	getSerialNumber	None
public	void	setSerialNumber	String serialNumber
public	String	getVersion	None
public	void	setVersion	String version
public	String	getProblemDescription	None
public	void	setProblemDescription	String problemDescription
public	String	getRepairDescription	None
public	void	setRepairDescription	String repairDescription
public	boolean	isReceived	None
public	void	setReceived	boolean received
public	String	toString	None

RepairItemSearchResult

Read-only projection used to display repair-item search matches with parent RMA context.

Package	com.harpenenterprises.rmatracker.model
Type	Class
Source file	com/harpenenterprises/rmatracker/model/RepairItemSearchResult.java
Extends	None

Implements	None
-------------------	------

Key fields

Visibility	Static	Type	Name
private	No	String	rmaNumber
private	No	RepairItem	repairItem

Constructors

- `public RepairItemSearchResult(String rmaNumber, RepairItem repairItem)`

Public and protected operations

Visibility	Return Type	Method	Parameters
public	String	getRmaNumber	None
public	RepairItem	getRepairItem	None

RmaRecord

Primary aggregate model representing an RMA and its dates, status, notes, repair items, and shipping records.

Package	com.harpenrprises.rmatracker.model
Type	Class
Source file	com/harpenrprises/rmatracker/model/RmaRecord.java
Extends	None
Implements	None

Key fields

Visibility	Static	Type	Name
private	No	String	rmaNumber
private	No	LocalDate	dateSent

private	No	LocalDate	dateReceived
private	No	Status	status
private	No	String	outgoingTrackingNumber
private	No	String	returnTrackingNumber
private	No	String	notes
private	No	List<RepairItem>	repairItems
private	No	List<ShippingInfo>	shippingInformation

Constructors

- public RmaRecord()
- public RmaRecord(String rmaNumber, LocalDate dateSent, LocalDate dateReceived, Status status, String outgoingTrackingNumber, String returnTrackingNumber, String notes)
- public RmaRecord(String rmaNumber, LocalDate dateSent, LocalDate dateReceived, Status status, String outgoingTrackingNumber, String returnTrackingNumber, String notes, List<RepairItem> repairItems)
- public RmaRecord(String rmaNumber, LocalDate dateSent, LocalDate dateReceived, Status status, String outgoingTrackingNumber, String returnTrackingNumber, String notes, List<RepairItem> repairItems, List<ShippingInfo> shippingInformation)

Public and protected operations

Visibility	Return Type	Method	Parameters
public	String	getRmaNumber	None
public	void	setRmaNumber	String rmaNumber
public	LocalDate	getDateSent	None
public	void	setDateSent	LocalDate dateSent
public	LocalDate	getDateReceived	None
public	void	setDateReceived	LocalDate dateReceived
public	Status	getStatus	None

Visibility	Return Type	Method	Parameters
public	void	setStatus	Status status
public	String	getOutgoingTrackingNumber	None
public	void	setOutgoingTrackingNumber	String value
public	String	getReturnTrackingNumber	None
public	void	setReturnTrackingNumber	String value
public	String	getNotes	None
public	void	setNotes	String notes
public	List<RepairItem>	getRepairItems	None
public	void	setRepairItems	List<RepairItem> repairItems
public	void	addRepairItem	RepairItem repairItem
public	boolean	removeRepairItem	RepairItem repairItem
public	List<ShippingInfo>	getShippingInformation	None
public	void	setShippingInformation	List<ShippingInfo> shippingInformation
public	void	addShippingInfo	ShippingInfo shippingInfo
public	boolean	removeShippingInfo	ShippingInfo shippingInfo

RmaSearchCriteria

Encapsulates optional search and filtering values used to query RMA data.

Package	com.harpenenterprises.rmatracker.model
Type	Class
Source file	com/harpenenterprises/rmatracker/model/RmaSearchCriteria.java

Extends	None
Implements	None

Key fields

Visibility	Static	Type	Name
private	No	String	searchText
private	No	Status	status
private	No	String	county
private	No	String	machineType
private	No	LocalDate	dateSentFrom
private	No	LocalDate	dateSentTo

Constructors

- public RmaSearchCriteria(String searchText, Status status, String county, String machineType, LocalDate dateSentFrom, LocalDate dateSentTo)

Public and protected operations

Visibility	Return Type	Method	Parameters
public	String	getSearchText	None
public	Status	getStatus	None
public	String	getCounty	None
public	String	getMachineType	None
public	LocalDate	getDateSentFrom	None
public	LocalDate	getDateSentTo	None

ShippingDirection

Enumeration distinguishing outbound and return shipment records.

Package	com.harperprises.rmatracker.model
Type	Enum
Source file	com/harperprises/rmatracker/model/ShippingDirection.java
Extends	None
Implements	None

Key fields

Visibility	Static	Type	Name
private	No	String	displayName

Public and protected operations

Visibility	Return Type	Method	Parameters
public	String	toString	None

ShippingInfo

Domain model for carrier, tracking number, and shipment direction information.

Package	com.harperprises.rmatracker.model
Type	Class
Source file	com/harperprises/rmatracker/model/ShippingInfo.java
Extends	None
Implements	None

Key fields

Visibility	Static	Type	Name
private	No	String	carrier
private	No	String	trackingNumber

private	No	ShippingDirection	direction
---------	----	-------------------	-----------

Constructors

- public ShippingInfo()
- public ShippingInfo(String carrier, String trackingNumber, ShippingDirection direction)

Public and protected operations

Visibility	Return Type	Method	Parameters
public	String	getCarrier	None
public	void	setCarrier	String carrier
public	String	getTrackingNumber	None
public	void	setTrackingNumber	String trackingNumber
public	ShippingDirection	getDirection	None
public	void	setDirection	ShippingDirection direction
public	String	toString	None
public	boolean	equals	Object object
public	int	hashCode	None

Status

Enumeration of supported RMA lifecycle states.

Package	com.harpenrprises.rmatracker.model
Type	Enum
Source file	com/harpenrprises/rmatracker/model/Status.java
Extends	None
Implements	None

The class exposes no separately detected public methods beyond constructors, inherited Swing behavior, or enum constants. Its work is performed through event listeners and internal helper methods.

StatusHistory

Represents a chronological status transition recorded for an RMA.

Package	com.harpenrprises.rmatracker.model
Type	Class
Source file	com/harpenrprises/rmatracker/model/StatusHistory.java
Extends	None
Implements	None

Key fields

Visibility	Static	Type	Name
private	No	long	id
private	No	String	rmaNumber
private	No	Status	oldStatus
private	No	Status	newStatus
private	No	LocalDateTime	changedAt

Constructors

- public StatusHistory()
- public StatusHistory(long id, String rmaNumber, Status oldStatus, Status newStatus, LocalDateTime changedAt)

Public and protected operations

Visibility	Return Type	Method	Parameters
public	long	getId	None
public	void	setId	long id
public	String	getRmaNumber	None
public	void	setRmaNumber	String rmaNumber
public	Status	getOldStatus	None
public	void	setOldStatus	Status oldStatus

Visibility	Return Type	Method	Parameters
public	Status	getNewStatus	None
public	void	setNewStatus	Status newStatus
public	LocalDateTime	getChangedAt	None
public	void	setChangedAt	LocalDateTime changedAt

6.6. Data Access Classes

SQLite connection, schema, and repository-based persistence components.

Type	Kind	Responsibility
DatabaseManager	Class	Owns SQLite connection creation, database path resolution, and schema initialization.
RmaRepository	Class	Primary persistence component for loading and saving RMA aggregates and related records.
StatusHistoryRepository	Class	Persistence component dedicated to inserting and retrieving chronological status history.

DatabaseManager

Owns SQLite connection creation, database path resolution, and schema initialization.

Package	com.harpenenterprises.rmatracker.storage
Type	Class
Source file	com/harpenenterprises/rmatracker/storage/DatabaseManager.java
Extends	None
Implements	None

Key fields

Visibility	Static	Type	Name
private	Yes	String	APPLICATION_FOLDER_NAME
private	Yes	String	DATABASE_FOLDER_NAME
private	Yes	String	DATABASE_FILE_NAME
private	Yes	File	APPLICATION_FOLDER

private	Yes	File	DATABASE_FOLDER
private	Yes	File	DATABASE_FILE
private	Yes	String	DATABASE_URL

Public and protected operations

Visibility	Return Type	Method	Parameters
public static	File	getDatabaseFile	None
public static	File	getApplicationFolder	None
public static	Connection	getConnection	None
public static	void	initializeDatabase	None

RmaRepository

Primary persistence component for loading and saving RMA aggregates and related records.

Package	com.harpenrprises.rmatracker.storage
Type	Class
Source file	com/harpenrprises/rmatracker/storage/RmaRepository.java
Extends	None
Implements	None

Public and protected operations

Visibility	Return Type	Method	Parameters
public	void	save	RmaRecord record
public	void	update	RmaRecord record
public	boolean	delete	String rmaNumber
public	Optional<RmaRecord>	findByRmaNumber	String rmaNumber
public	List<RmaRecord>	findAll	None

Visibility	Return Type	Method	Parameters
public	List<RmaRecord>	search	String text
public	boolean	existsByRmaNumber	String number

StatusHistoryRepository

Persistence component dedicated to inserting and retrieving chronological status history.

Package	com.harpenrprises.rmatracker.storage
Type	Class
Source file	com/harpenrprises/rmatracker/storage/StatusHistoryRepository.java
Extends	None
Implements	None

Public and protected operations

Visibility	Return Type	Method	Parameters
public	List<StatusHistory>	findByRmaNumber	String rmaNumber
public	boolean	deleteById	long historyId

6.7. Service Classes

Business and infrastructure services for search, export, and backup operations.

Type	Kind	Responsibility
BackupService	Class	Creates automatic and manual database backups and supports controlled restoration.
ExcelExportService	Class	Generates Microsoft Excel workbooks from selected or filtered RMA records.

Type	Kind	Responsibility
PdfExportService	Class	Transforms selected or filtered RMA data into portable PDF documents.
RmaSearchService	Class	Business service that applies free-text and criteria-based searching across RMA and repair-item data.

BackupService

Creates automatic and manual database backups and supports controlled restoration.

Package	com.harpenenterprises.rmatracker.service
Type	Class
Source file	com/harpenenterprises/rmatracker/service/BackupService.java
Extends	None
Implements	None

Key fields

Visibility	Static	Type	Name
private	Yes	DateTimeFormatter	FILE_TIMESTAMP
private	Yes	byte[]	SQLITE_HEADER

Public and protected operations

Visibility	Return Type	Method	Parameters
public	Path	createBackup	Path destinationFolder
public	Path	restoreBackup	Path selectedBackup
public	void	validateBackup	Path backupPath

ExcelExportService

Generates Microsoft Excel workbooks from selected or filtered RMA records.

Package	com.harpenrprises.rmatracker.service
Type	Class
Source file	com/harpenrprises/rmatracker/service/ExcelExportService.java
Extends	None
Implements	None

Key fields

Visibility	Static	Type	Name
private	Yes	DateTimeFormatter	GENERATED_TIME_FORMAT
private	Yes	String[]	SELECTED_RMA_HEADERS
private	Yes	String[]	DISPLAYED_RMA_HEADERS

Public and protected operations

Visibility	Return Type	Method	Parameters
public	void	exportSelectedRma	RmaRecord record, Path outputFile
public	void	exportDisplayedRmas	List<RmaRecord> records, Path outputFile

PdfExportService

Transforms selected or filtered RMA data into portable PDF documents.

Package	com.harpenrprises.rmatracker.service
Type	Class
Source file	com/harpenrprises/rmatracker/service/PdfExportService.java
Extends	None
Implements	None

Key fields

Visibility	Static	Type	Name
private	Yes	DateTimeFormatter	GENERATED_FORMAT
private	Yes	float	PAGE_WIDTH
private	Yes	float	PAGE_HEIGHT
private	Yes	float	LEFT
private	Yes	float	RIGHT
private	Yes	float	TOP
private	Yes	float	BOTTOM
private	Yes	float	ROW_HEIGHT
private	Yes	String[]	HEADERS
private	Yes	float[]	WIDTHS
private	No	List<StringBuilder>	pages
private	No	StringBuffer	stream
private	No	float	y

Public and protected operations

Visibility	Return Type	Method	Parameters
public	void	exportSelectedRma	RmaRecord record, Path outputFile
public	void	exportDisplayedRmas	List<RmaRecord> records, Path outputFile

RmaSearchService

Business service that applies free-text and criteria-based searching across RMA and repair-item data.

Package	com.harpenenterprises.rmatracker.service
Type	Class
Source file	com/harpenenterprises/rmatracker/service/RmaSearchService.java
Extends	None
Implements	None

Public and protected operations

Visibility	Return Type	Method	Parameters
public	List<RmaRecord>	search	List<RmaRecord> records, RmaSearchCriteria criteria

6.8. Reporting Classes

Report-construction components shared by preview and print workflows.

Type	Kind	Responsibility
RmaReportHtmlBuilder	Class	Builds HTML-based report content used by report preview and printing workflows.

RmaReportHtmlBuilder

Builds HTML-based report content used by report preview and printing workflows.

Package	com.harpenenterprises.rmatracker.report
Type	Class
Source file	com/harpenenterprises/rmatracker/report/RmaReportHtmlBuilder.java

Extends	None
Implements	None

Key fields

Visibility	Static	Type	Name
private	Yes	DateTimeFormatter	DATE_FORMATTER
private	Yes	DateTimeFormatter	HISTORY_DATE_FORMATTER

Public and protected operations

Visibility	Return Type	Method	Parameters
public	String	build	RmaRecord record
public	String	build	RmaRecord record, List<StatusHistory> statusHistory

6.9. User Interface Classes

Swing windows, dialogs, panels, menus, and action components.

Type	Kind	Responsibility
MainWindow	Class	Primary application frame coordinating RMA lists, details, menus, actions, searching, and refresh operations.
MainWindowActionBar	Class	Toolbar component containing high-frequency commands for record and report operations.
MainWindowDetailsPanel	Class	Detail presentation component for repair items and selected-RMA information.

Type	Kind	Responsibility
MainWindowMenuBar	Class	Application menu bar that exposes file, record, reporting, backup, and help commands.
MainWindowSearchPanel	Class	Search and status-filter controls used to refine the records displayed in the main window.
RmaFormActionBar	Class	Dialog action area containing save, cancel, and related controls.
RmaFormDialog	Class	Modal create/edit dialog that coordinates RMA information, repair items, shipping data, validation, and save operations.
RmaFormInformationPanel	Class	Form subsection containing core RMA identifiers, dates, status, and notes.
RmaFormRepairItemsPanel	Class	Form subsection for adding, editing, and removing repair-item rows.
RmaFormShippingPanel	Class	Form subsection for managing multiple outbound and return shipment entries.
RmaPrintPreviewWindow	Class	Displays printable list content and provides page-oriented printing actions.
RmaReportActionBar	Class	Action controls associated with report preview, print, PDF, and Excel workflows.

Type	Kind	Responsibility
RmaReportPreviewWindow	Class	Displays a formatted selected-RMA report before print or export.
StatusHistoryDialog	Class	Displays the chronological status history for the selected RMA.

MainWindow

Primary application frame coordinating RMA lists, details, menus, actions, searching, and refresh operations.

Package	com.harpenenterprises.rmatracker.ui
Type	Class
Source file	com/harpenenterprises/rmatracker/ui/MainWindow.java
Extends	JFrame
Implements	None

Key fields

Visibility	Static	Type	Name
private	Yes	int	MAX_AUTOMATIC_BACKUPS
private	Yes	Path	AUTOMATIC_BACKUP_FOLDE R
private	No	TextField	searchField
private	No	TextField	dateFromField
private	No	TextField	dateToField
private	No	JComboBox<Object>	statusFilterComboBox
private	No	JComboBox<String>	countyFilterComboBox
private	No	JComboBox<String>	machineFilterComboBox

private	No	JButton	clearSearchButton
private	No	JTable	rmaTable
private	No	DefaultTableModel	rmaTableModel
private	No	TableRowSorter<DefaultTableModel >	rmaTableSorter
private	No	JTable	repairItemsTable
private	No	DefaultTableModel	repairItemsTableModel
private	No	JLabel	selectedRmaNumberValue
private	No	JLabel	selectedDateSentValue
private	No	JLabel	selectedDateReceivedValue
private	No	JLabel	selectedStatusValue

Constructors

- public MainWindow()

Public and protected operations

Visibility	Return Type	Method	Parameters
public	boolean	isCellEditable	int row, int column
public	boolean	isCellEditable	int row, int column
public	void	windowClosing	WindowEvent event
public	void	insertUpdate	DocumentEvent event
public	void	removeUpdate	DocumentEvent event
public	void	changedUpdate	DocumentEvent event
public	void	mouseClicked	MouseEvent event
public	void	mouseClicked	MouseEvent event
public	void	actionPerformed	java.awt.event.ActionEvent event
public	void	actionPerformed	java.awt.event.ActionEvent event

Visibility	Return Type	Method	Parameters
public	Component	getTableCellRendererComponent	JTable table, Object value, boolean isSelected, boolean hasFocus, int row, int column
public	Component	getTableCellRendererComponent	JTable table, Object value, boolean isSelected, boolean hasFocus, int row, int column

MainWindowActionBar

Toolbar component containing high-frequency commands for record and report operations.

Package	com.harpenrprises.rmatracker.ui
Type	Class
Source file	com/harpenrprises/rmatracker/ui/MainWindowActionBar.java
Extends	JPanel
Implements	None

Constructors

- public MainWindowActionBar(JButton newRmaButton, JButton openRmaButton, JButton viewReportButton, JButton statusHistoryButton, JButton deleteRmaButton, JButton refreshButton, JButton exitButton, JLabel resultLabel)

The class exposes no separately detected public methods beyond constructors, inherited Swing behavior, or enum constants. Its work is performed through event listeners and internal helper methods.

MainWindowDetailsPanel

Detail presentation component for repair items and selected-RMA information.

Package	com.harpenrprises.rmatracker.ui
Type	Class
Source file	com/harpenrprises/rmatracker/ui/MainWindowDetailsPanel.java

Extends	JPanel
Implements	None

Constructors

- public MainWindowDetailsPanel(JLabel detailMessageLabel, JLabel selectedRmaNumberValue, JLabel selectedDateSentValue, JLabel selectedDateReceivedValue, JLabel selectedStatusValue, JTextArea selectedSentShippingValue, JTextArea selectedReturnShippingValue, JTextArea selectedNotesArea, JTable repairItemsTable)

The class exposes no separately detected public methods beyond constructors, inherited Swing behavior, or enum constants. Its work is performed through event listeners and internal helper methods.

MainWindowMenuBar

Application menu bar that exposes file, record, reporting, backup, and help commands.

Package	com.harpenenterprises.rmatracker.ui
Type	Class
Source file	com/harpenenterprises/rmatracker/ui/MainWindowMenuBar.java
Extends	JMenuBar
Implements	None

Constructors

- public MainWindowMenuBar(Runnable backupAction, Runnable restoreAction, Runnable exitAction, Runnable viewSelectedReportAction, Runnable exportSelectedExcelAction, Runnable exportDisplayedExcelAction, Runnable exportSelectedPdfAction, Runnable exportDisplayedPdfAction)

The class exposes no separately detected public methods beyond constructors, inherited Swing behavior, or enum constants. Its work is performed through event listeners and internal helper methods.

MainWindowSearchPanel

Search and status-filter controls used to refine the records displayed in the main window.

Package	com.harpenenterprises.rmatracker.ui
----------------	-------------------------------------

Type	Class
Source file	com/harpenterprises/rmatracker/ui/MainWindowSearchPanel.java
Extends	JPanel
Implements	None

Constructors

- `public MainWindowSearchPanel(JTextField searchField, JButton clearSearchButton, JComboBox<Object> statusFilterComboBox, JComboBox<String> countyFilterComboBox, JComboBox<String> machineFilterComboBox, JTextField dateFromField, JTextField dateToField)`

The class exposes no separately detected public methods beyond constructors, inherited Swing behavior, or enum constants. Its work is performed through event listeners and internal helper methods.

RmaFormActionBar

Dialog action area containing save, cancel, and related controls.

Package	com.harpenterprises.rmatracker.ui
Type	Class
Source file	com/harpenterprises/rmatracker/ui/RmaFormActionBar.java
Extends	JPanel
Implements	None

Constructors

- `public RmaFormActionBar(JButton saveButton, JButton cancelButton)`

The class exposes no separately detected public methods beyond constructors, inherited Swing behavior, or enum constants. Its work is performed through event listeners and internal helper methods.

RmaFormDialog

Modal create/edit dialog that coordinates RMA information, repair items, shipping data, validation, and save operations.

Package	com.harpenterprises.rmatracker.ui
Type	Class
Source file	com/harpenterprises/rmatracker/ui/RmaFormDialog.java
Extends	JDialog
Implements	None

Key fields

Visibility	Static	Type	Name
private	No	TextField	rmaNumberField
private	No	TextField	dateSentField
private	No	TextField	dateReceivedField
private	No	JComboBox<Status>	statusComboBox
private	No	TextField	outgoingTrackingField
private	No	TextField	returnTrackingField
private	No	TextArea	notesArea
private	No	JTable	repairItemsTable
private	No	DefaultTableModel	repairItemsTableModel
private	No	JTable	shippingTable
private	No	DefaultTableModel	shippingTableModel
private	No	JButton	addItemButton
private	No	JButton	removeItemButton
private	No	JButton	addShippingButton
private	No	JButton	removeShippingButton

private	No	JButton	saveButton
private	No	JButton	cancelButton
private	No	RmaRepository	repository

Constructors

- public RmaFormDialog(JFrame parent)
- public RmaFormDialog(JFrame parent, RmaRecord existingRecord)

Public and protected operations

Visibility	Return Type	Method	Parameters
public	Class<?>	getColumnClass	int columnIndex
public	Class<?>	getColumnClass	int columnIndex
public	void	windowClosing	WindowEvent event
public	boolean	isSaved	None
public	void	insertUpdate	DocumentEvent event
public	void	removeUpdate	DocumentEvent event
public	void	changedUpdate	DocumentEvent event
public	void	actionPerformed	java.awt.event.ActionEvent event
public	void	actionPerformed	java.awt.event.ActionEvent event

RmaFormInformationPanel

Form subsection containing core RMA identifiers, dates, status, and notes.

Package	com.harpenenterprises.rmatracker.ui
Type	Class
Source file	com/harpenenterprises/rmatracker/ui/RmaFormInformationPanel.java
Extends	JPanel

Implements	None
-------------------	------

Constructors

- `public RmaFormInformationPanel(JTextField rmaNumberField, JTextField dateSentField, JTextField dateReceivedField, JComboBox<Status> statusComboBox, JTextArea notesArea)`

The class exposes no separately detected public methods beyond constructors, inherited Swing behavior, or enum constants. Its work is performed through event listeners and internal helper methods.

RmaFormRepairItemsPanel

Form subsection for adding, editing, and removing repair-item rows.

Package	com.harpenterprises.rmatracker.ui
Type	Class
Source file	com/harpenterprises/rmatracker/ui/RmaFormRepairItemsPanel.java
Extends	JPanel
Implements	None

Constructors

- `public RmaFormRepairItemsPanel(JTable repairItemsTable, JButton addItemButton, JButton removeItemButton)`

The class exposes no separately detected public methods beyond constructors, inherited Swing behavior, or enum constants. Its work is performed through event listeners and internal helper methods.

RmaFormShippingPanel

Form subsection for managing multiple outbound and return shipment entries.

Package	com.harpenterprises.rmatracker.ui
Type	Class
Source file	com/harpenterprises/rmatracker/ui/RmaFormShippingPanel.java
Extends	JPanel
Implements	None

Constructors

- `public RmaFormShippingPanel(JTable shippingTable, JButton addShippingButton, JButton removeShippingButton)`

The class exposes no separately detected public methods beyond constructors, inherited Swing behavior, or enum constants. Its work is performed through event listeners and internal helper methods.

RmaPrintPreviewWindow

Displays printable list content and provides page-oriented printing actions.

Package	com.harpenrprises.rmatracker.ui
Type	Class
Source file	com/harpenrprises/rmatracker/ui/RmaPrintPreviewWindow.java
Extends	JDialog
Implements	None

Key fields

Visibility	Static	Type	Name
private	Yes	DateTimeFormatter	DATE_FORMATTER
private	Yes	DateTimeFormatter	HISTORY_DATE_FORMATTER
private	Yes	double	POINTS_PER_INCH
private	Yes	double	DEFAULT_MARGIN
private	Yes	double	HEADER_HEIGHT
private	Yes	double	FOOTER_HEIGHT
private	Yes	int	MACHINES_PER_PAGE
private	Yes	int	HISTORY_ROWS_PER_PAGE
private	No	RmaRecord	rmaRecord
private	No	StatusHistoryRepository	statusHistoryRepository
private	No	JEditorPane	pagePane
private	No	PreviewCanvas	previewCanvas

private	No	JLabel	pageLabel
private	No	JButton	previousButton
private	No	JButton	nextButton
private	No	JComboBox<String>	zoomComboBox
private	No	List<String>	pageHtml
private	No	PageFormat	pageFormat

Constructors

- public RmaPrintPreviewWindow(Window owner, RmaRecord rmaRecord)

Public and protected operations

Visibility	Return Type	Method	Parameters
protected	void	paintComponent	Graphics graphics

RmaReportActionBar

Action controls associated with report preview, print, PDF, and Excel workflows.

Package	com.harpenrprises.rmatracker.ui
Type	Class
Source file	com/harpenrprises/rmatracker/ui/RmaReportActionBar.java
Extends	JPanel
Implements	None

Constructors

- public RmaReportActionBar(Runnable refreshAction, Runnable printAction, Runnable savePdfAction, Runnable closeAction)

The class exposes no separately detected public methods beyond constructors, inherited Swing behavior, or enum constants. Its work is performed through event listeners and internal helper methods.

RmaReportPreviewWindow

Displays a formatted selected-RMA report before print or export.

Package	com.harpenrprises.rmatracker.ui
Type	Class
Source file	com/harpenrprises/rmatracker/ui/RmaReportPreviewWindow.java
Extends	JDialog
Implements	None

Key fields

Visibility	Static	Type	Name
private	Yes	DateTimeFormatter	DATE_FORMATTER
private	Yes	DateTimeFormatter	HISTORY_DATE_FORMATTER
private	Yes	double	POINTS_PER_INCH
private	Yes	double	DEFAULT_MARGIN
private	Yes	double	HEADER_HEIGHT
private	Yes	double	FOOTER_HEIGHT
private	No	RmaRecord	rmaRecord
private	No	JEditorPane	reportPane
private	No	StatusHistoryRepository	statusHistoryRepository
private	No	PageFormat	pageFormat

Constructors

- public RmaReportPreviewWindow(Window owner, RmaRecord rmaRecord)

The class exposes no separately detected public methods beyond constructors, inherited Swing behavior, or enum constants. Its work is performed through event listeners and internal helper methods.

StatusHistoryDialog

Displays the chronological status history for the selected RMA.

Package	com.harpenenterprises.rmatracker.ui
Type	Class
Source file	com/harpenenterprises/rmatracker/ui/StatusHistoryDialog.java
Extends	JDialog
Implements	None

Key fields

Visibility	Static	Type	Name
private	Yes	DateTimeFormatter	DATE_FORMATTER
private	No	String	rmaNumber
private	No	StatusHistoryRepository	repository
private	No	DefaultTableModel	tableModel
private	No	JTable	historyTable
private	No	JButton	deleteButton
private	No	List<StatusHistory>	displayedHistory

Constructors

- public StatusHistoryDialog(Window owner, String rmaNumber)

Public and protected operations

Visibility	Return Type	Method	Parameters
public	boolean	isCellEditable	int row, int column

6.10. Utility Classes

Reusable, application-wide helper utilities.

Type	Kind	Responsibility
DateUtil	Class	Central date parsing and formatting helper used across UI, persistence, and reporting code.

DateUtil

Central date parsing and formatting helper used across UI, persistence, and reporting code.

Package	com.harpenenterprises.rmatracker.util
Type	Class
Source file	com/harpenenterprises/rmatracker/util/DateUtil.java
Extends	None
Implements	None

The class exposes no separately detected public methods beyond constructors, inherited Swing behavior, or enum constants. Its work is performed through event listeners and internal helper methods.

6.11. Exception and Error Propagation

The current implementation primarily uses standard Java and SQL exceptions rather than a large custom exception hierarchy. Repository and service classes are responsible for catching low-level failures at appropriate boundaries, preserving the original cause for diagnostics, and presenting user-safe messages through the Swing interface. This avoids exposing JDBC, file-system, or export-library details directly to end users.

Recommended future improvement: introduce focused application exceptions such as DatabaseOperationException, ExportException, BackupException, and ValidationException when the codebase grows enough to justify consistent cross-layer error contracts.

6.12. Class Relationships

Source	Relationship	Target
Main	creates and displays	MainWindow
MainWindow	loads, refreshes, and modifies RMA data	RmaRepository
MainWindow	filters visible records	RmaSearchService
MainWindow	opens create and edit workflows	RmaFormDialog
RmaFormDialog	collects and updates aggregate data	RmaRecord
RmaRepository	obtains SQLite connections	DatabaseManager
RmaRepository	maps database rows to aggregates	RmaRecord
RmaRecord	contains zero or more	RepairItem
RmaRecord	contains zero or more	ShippingInfo
StatusHistoryRepository	persists and loads	StatusHistory
RmaReportPreviewWindow	builds preview content	RmaReportHtmlBuilder
PdfExportService	exports	RmaRecord
ExcelExportService	exports	RmaRecord
BackupService	protects the active database file	DatabaseManager

6.13. Method-Level Reference Guidance

The method tables in this chapter provide a maintainable public-interface reference rather than a substitute for Javadoc. Developers should preserve method names that clearly communicate intent, keep UI event handlers short, place database SQL in repository classes, and place reusable export, backup, and search behavior in services. Significant public methods should receive Javadoc describing parameters, return values, side effects, exceptions, and thread-safety expectations.

6.14. Extension Points

- New domain entities should be added to the model package and persisted through dedicated repository methods or repositories.
- New export formats should be implemented as service classes rather than added directly to MainWindow.

New RMA form sections should follow the existing panel decomposition used by RmaFormDialog.

Authentication and authorization should be introduced through service interfaces so UI components do not own security rules.

Cloud synchronization should be isolated behind a synchronization service and should not replace repository abstractions.

Additional reports should reuse shared report builders and action bars where practical.

Database schema changes should be paired with controlled migration logic and updated model/repository documentation.

7. User Interface Design

The RMA Tracker graphical user interface provides a desktop environment for creating, editing, searching, tracking, reporting, and maintaining Return Merchandise Authorization (RMA) records. The interface is organized into reusable windows, dialogs, menus, tables, and panels that separate presentation logic from business logic and persistence. The design emphasizes consistency, ease of navigation, data validation, responsiveness, and maintainability.

7.1. UI Design Overview

The interface follows a modular Swing architecture. Each major screen is implemented as an independent class, allowing future enhancements without affecting unrelated functionality. Common validation routines, table models, dialogs, and event handlers are reused throughout the application.

7.2. Main Dashboard

The Main Dashboard serves as the application's central workspace. It provides access to all RMA records, search and filtering tools, action buttons, menus, reporting functions, backup utilities, and detailed record views. Double-clicking an RMA opens the edit dialog.

7.3. Create RMA Window

The Create RMA dialog collects all information required to create a new RMA, including customer information, dates, repair items, shipping details, notes, and status. Required fields are validated before saving.

7.4. Edit RMA Window

The Edit dialog retrieves an existing RMA from the database, populates all controls, validates user modifications, and updates the database upon confirmation.

7.5. Search and Filtering

Search supports RMA number, county, serial number, tracking number, machine type, repair descriptions, notes, and status. Filtered results refresh immediately.

7.6. Repair Item Management

Each RMA may contain multiple repair items. Users may add, edit, delete, and mark items received. Table columns include county, serial number, equipment type, version, problem description, repair description, and received status.

7.7. Shipping Interface

Shipping records support multiple carrier entries, outbound and return tracking numbers, shipment direction, and historical tracking information.

7.8. Status History Interface

Every status transition is permanently recorded with timestamps, previous status, new status, and user-generated comments where applicable.

7.9. Report Preview

The report preview window displays printable reports before printing or PDF generation. Users can preview, print, export, and verify formatting.

7.10. Backup and Restore Windows

Dedicated dialogs provide manual backup creation, automatic backup verification, restore confirmation, and database recovery.

7.11. Settings Window

The settings interface centralizes application preferences such as default folders, automatic backup behavior, future themes, and application options.

7.12. Menus and Keyboard Shortcuts

The menu system provides File, Edit, Reports, Backup, Export, Help, and maintenance commands with standard keyboard accelerators.

7.13. UI Validation and Error Messages

Validation prevents incomplete or invalid records. Errors are presented through descriptive dialog boxes that guide users toward corrective action.

7.14. Accessibility and Usability

Consistent spacing, readable fonts, logical tab ordering, keyboard accessibility, confirmation dialogs, and standardized layouts improve usability.

7.15. UI Component Diagrams

The final manual should include navigation maps, window hierarchy diagrams, panel relationships, sequence diagrams, and annotated screenshots for every major interface.

Major User Interface Classes

Class	Purpose	Primary Responsibilities
MainWindow	Application Shell	Dashboard, navigation, search, menus
RmaFormDialog	Create/Edit Dialog	Data entry and validation
StatusHistoryDialog	History Viewer	Display chronological status changes
RmaReportPreviewWindow	Reporting	Preview and print reports
BackupService UI	Maintenance	Backup and restore workflows

Design Principles

- Consistent window layouts.
- Reusable dialog components.
- Clear separation between UI, services, and persistence.
- Immediate validation feedback.
- Support for large RMA datasets.
- Keyboard-friendly navigation.
- Extensible architecture for Version 3.0.

8. Reporting System

The reporting subsystem converts live RMA data into printable and exportable business documents. It supports report preview, PDF generation, Excel export, and printing while maintaining consistent formatting and ensuring that reports accurately reflect the current database state.

8.1. Reporting Architecture

Reporting components retrieve data from the repository layer, format it into business objects, and render printable output.

8.2. Report Data Sources

Reports include RMA records, repair items, shipping information, status history, and notes from the SQLite database.

8.3. Report Templates

Templates define page headers, company information, machine tables, summaries, and footer formatting.

8.4. Report Preview

Preview windows render reports before printing or exporting, allowing users to verify formatting.

8.5. PDF Export

Selected RMAs and filtered search results may be exported to PDF using standardized layouts and filenames.

8.6. Excel Export

Reports may also be exported as XLSX workbooks with one repair item per row for further analysis.

8.7. Printing

The printing subsystem supports page setup, printer selection, scaling, and multi-page reports.

8.8. File Naming Conventions

Generated files include the report type, RMA number when applicable, and timestamp to ensure uniqueness.

8.9. Reporting Error Handling

Exceptions during preview, export, or printing are presented through descriptive dialog boxes and logged.

8.10. Future Reporting Enhancements

Future versions will include scheduled reports, dashboard analytics, custom templates, email delivery, and cloud reporting.

9. Backup and Recovery System

The backup and recovery subsystem protects application data from accidental deletion, corruption, hardware failure, and software errors. It provides both automatic and manual backup capabilities together with guided restoration procedures.

9.1. Backup Architecture

Backup services operate independently from the user interface and copy the SQLite database to protected backup folders.

9.2. Automatic Backups

An automatic backup is created when the application exits successfully, providing continuous protection.

9.3. Manual Backups

Users may create on-demand backups before upgrades or major edits.

9.4. Backup File Locations

Backups are stored in application support directories with separate automatic and manual folders.

9.5. Backup Naming and Rotation

Backup filenames contain timestamps. Older backups may be retained or removed according to future retention policies.

9.6. Restore Process

Users select a backup, confirm the operation, and replace the active database after validation.

9.7. Validation and Safety Checks

The application verifies backup existence and prompts users before overwriting production data.

9.8. Recovery from Failure

Recovery procedures restore operations following database corruption, accidental deletion, or failed updates.

9.9. Backup Logging

Backup and restore operations record success, failure, timestamps, and diagnostic information.

9.10. Backup Security

Backup files rely on operating system permissions and local storage protection.

9.11. Future Cloud Backup

Future releases will support encrypted cloud synchronization, remote recovery, and version history.

10. Packaging and Distribution

This chapter documents the processes, tools, standards, and deployment procedures used to build, package, distribute, install, and maintain RMA Tracker Version 2.1.1. The objective is to produce repeatable, reliable releases across supported desktop operating systems while preserving application integrity, database compatibility, and user configuration.

10.1. Packaging Overview

- Packaging goals include repeatable builds, consistent versioning, simple installation, protection of user data, and straightforward upgrades.
- Primary build tool: Gradle.
- Primary runtime: Java 17 or later.
- Current target platform: macOS, with Windows and Linux planned.
- Deliverables include executable JARs, native application bundles, installers, documentation, release notes, and archived source code.

10.2. Gradle Build Process

- Gradle automates compilation, dependency resolution, testing, packaging, and artifact generation.
- Typical lifecycle: clean → compile → test → jar → jpackage.
- Recommended commands: gradlew clean, gradlew build, gradlew test.
- All successful builds should be archived with version numbers and release notes.

10.3. Executable JAR

- The executable JAR contains compiled classes, resources, and dependency metadata.
- The manifest defines the application's main entry point.
- The executable JAR is primarily intended for development, testing, and troubleshooting.

10.4. jpackage Configuration

- Native installers are generated using jpackage.
- Configuration includes application name, version, vendor, icon, runtime image, destination folder, and installer options.
- Platform-specific icons should be provided for macOS, Windows, and Linux.

10.5. macOS Application Bundle

- The macOS build produces a standard .app bundle containing the runtime image, executable, libraries, resources, and configuration.
- User data is stored within the Application Support directory rather than inside the application bundle.

10.6. macOS DMG

- The DMG provides an installer-friendly distribution image. Users install by dragging the application into the Applications folder.
- Future releases should include notarization and code signing.

10.7. Windows Packaging

- Future Windows releases should generate MSI and EXE installers with Start Menu shortcuts, uninstall support, and optional desktop shortcuts.

10.8. Linux Packaging

- Future Linux releases should support AppImage, DEB, and RPM distributions where practical.

10.9. Application Versioning

- RMA Tracker follows Semantic Versioning (Major.Minor.Patch).
- Patch releases correct defects; minor releases add compatible functionality; major releases introduce architectural changes.

10.10. Distribution Structure

- Each release package should contain the executable, documentation, release notes, license, and checksum information.
- Source archives should be distributed separately from production installers.

10.11. Code Signing and Notarization

- Future production releases should be digitally signed to improve operating-system trust and installation security.
- macOS notarization and Windows Authenticode certificates are recommended.

10.12. Packaging Troubleshooting

- Common issues include incorrect runtime versions, missing dependencies, invalid icons, path problems, and permission errors.
- Recommended troubleshooting begins with Gradle logs followed by verification of runtime configuration, packaging commands, and application resources.

Release Checklist

- Run all automated tests successfully.
- Verify database schema compatibility.

- Confirm report generation and exports.
- Validate backup and restore functionality.
- Package native installers.
- Verify installation on a clean machine.
- Update documentation and release notes.
- Archive build artifacts.

Recommended Distribution Layout

Folder/File	Purpose
Application	Native installer or application bundle
Documentation	Volumes I–IV manuals
Release Notes	Version changes
License	Software license
Checksums	Integrity verification
Source Archive	Tagged source code

Best Practices

Every release should be built from a tagged source revision, tested before packaging, assigned a semantic version, archived, and accompanied by updated documentation. Database backups should always be created before upgrades. Installer verification should be performed on a clean system to ensure that hidden development dependencies have not been introduced.

11. Testing Strategy and Quality Assurance

This chapter defines the testing philosophy, quality assurance practices, and verification procedures used throughout the development of RMA Tracker. The objective is to ensure correctness, reliability, maintainability, and a consistent user experience.

11.1. Testing Overview

Testing is performed throughout development using a combination of unit, integration, system, and manual testing. Every change should be verified before release.

11.2. Unit Testing

Business logic, models, repositories, utilities, and services should be validated using JUnit. Tests verify expected inputs, outputs, edge cases, and exception handling.

11.3. Integration Testing

Integration tests confirm interaction between the Swing interface, service layer, SQLite database, reporting components, and backup subsystem.

11.4. Database Testing

CRUD operations, schema initialization, constraints, foreign-key relationships, searches, and transactions are verified using representative datasets.

11.5. User Interface Testing

Every dialog, menu, toolbar action, table, search filter, and validation message is manually verified. Keyboard navigation, focus order, resizing, and usability are also reviewed.

11.6. Reporting Tests

PDF generation, Excel export, report preview, print formatting, pagination, and file naming conventions are validated.

11.7. Backup and Restore Tests

Automatic backups, manual backups, restore operations, recovery after corruption, and backup integrity are verified before release.

11.8. Packaging Tests

Executable JARs, macOS application bundles, DMG installers, and future Windows installers are tested on clean environments.

11.9. Regression Testing

Previously implemented features are retested after every modification to ensure no existing functionality is broken.

11.10. Manual Test Procedures

A documented checklist is followed covering installation, creating RMAs, editing, searching, reporting, exporting, backup, restore, and application shutdown.

11.11. Test Data

Test databases include small, medium, and large RMA datasets with diverse repair items, shipping records, and status histories.

11.12. Defect Tracking

Defects are recorded with severity, reproduction steps, screenshots, root cause, resolution, and verification status.

11.13. Release Readiness Criteria

All critical defects resolved, automated tests passing, documentation updated, packaging verified, and release approved.

11.14. Test Results and Evidence

Evidence includes screenshots, console logs, exported reports, test matrices, and signed release checklists.

12.Security, Reliability, and Performance Design

This chapter documents the security model, reliability objectives, and performance considerations of RMA Tracker. As a desktop application, the system focuses on protecting local data, validating input, and maintaining operational stability.

12.1. Current Security Model

The application operates locally without network exposure. Access is controlled primarily through operating-system user accounts.

12.2. Data Protection

The SQLite database, exported reports, and backup files are stored in application support directories. Users are encouraged to protect these locations using operating-system permissions and regular backups.

12.3. Input Validation

All user input is validated before persistence. Required fields, dates, identifiers, and table entries are checked to reduce invalid data.

12.4. Error Handling Strategy

Exceptions are caught, logged where appropriate, and displayed using user-friendly dialog messages. Unexpected failures should never silently corrupt data.

12.5. Reliability Design

Modular architecture, repository abstraction, automatic backups, and transactional database updates improve long-term reliability.

12.6. Performance Goals

The application is designed to launch quickly, search large RMA collections efficiently, and generate reports within acceptable desktop response times.

12.7. Database Performance

Indexes, optimized SQL queries, and normalized tables support responsive searches and updates.

12.8. UI Responsiveness

Long-running operations should provide progress feedback and avoid freezing the user interface.

12.9. Logging and Diagnostics

Diagnostic logging assists troubleshooting while avoiding unnecessary exposure of sensitive information.

12.10. Future Authentication and Authorization

Future releases will introduce user accounts, role-based permissions, audit trails, password hashing, and optional multi-factor authentication.

13.. Maintenance and Future Architecture

This chapter documents the long-term maintenance strategy and architectural roadmap for RMA Tracker. Its purpose is to provide future developers and maintainers with guidance for extending the application while preserving stability, maintainability, and backward compatibility. The architecture emphasizes modular design, clear separation of responsibilities, reusable components, and incremental evolution toward an enterprise-ready platform.

13.1. Maintenance Overview

Maintenance activities include defect correction, dependency updates, documentation revisions, performance improvements, and feature enhancements. Every modification should be reviewed, tested, documented, and versioned before release.

13.2. Adding New Features

New functionality should be implemented using the existing layered architecture. New features should minimize coupling, maximize reuse, and avoid breaking existing workflows. All new UI components, services, repositories, and models should follow established naming conventions.

13.3. Modifying the Database

Database schema changes should be backward compatible whenever possible. Schema migrations should be version-controlled, tested with representative data, and accompanied by migration scripts and updated documentation.

13.4. Adding New Reports

New reports should use the existing reporting framework and formatting conventions. Report templates should separate data retrieval from presentation to simplify maintenance and support future export formats.

13.5. Updating Dependencies

Third-party libraries should be reviewed regularly for bug fixes, security updates, and compatibility with the current Java version. Dependency upgrades should be validated using regression testing before release.

13.6. Refactoring Guidance

Refactoring should improve readability, reduce duplication, simplify class responsibilities, and maintain behavior. Large refactoring efforts should be completed incrementally and verified with automated and manual testing.

13.7. Planned Version 2.2 Architecture

Planned Version 2.2 enhancements include improved reporting, expanded search capabilities, additional configuration options, enhanced backup management, user interface refinements, and continued codebase cleanup.

13.8. Planned Version 3.0 Architecture

Version 3.0 represents the transition toward a configurable service-management platform supporting user authentication, configurable business rules, modular extensions, enhanced reporting, dashboard analytics, and optional cloud synchronization.

13.9. Cloud Synchronization

Future releases may synchronize databases securely between desktop installations using encrypted cloud storage while maintaining offline operation and conflict resolution.

13.10. User Accounts and Roles

Role-based authentication will introduce administrators, managers, technicians, and read-only users. Permissions will control access to configuration, reporting, backups, and administrative functions.

13.11. Automatic Tracking Updates

Future integration with shipping carrier APIs will automatically retrieve shipment status, reducing manual updates and improving tracking accuracy.

13.12. API and Integration Layer

A REST API will allow external systems to exchange RMAs, reports, customers, and inventory information with the application. Authentication, rate limiting, and versioning will be incorporated into the API design.

13.13. Scalability Considerations

The architecture is intended to evolve from a single-user desktop application into a configurable platform capable of supporting larger datasets, multi-user deployments, cloud services, and future WarpTech modules.

Future Roadmap Summary

Version	Primary Goals	Representative Features
2.2	Usability and refinement	Enhanced reporting, search improvements, UI polish
3.0	Enterprise foundation	Authentication, dashboards, cloud sync, APIs
Future Platform	Configurable business platform	Inventory, CRM, customer portal, mobile support, AI assistant

14.. Appendices

The appendices provide detailed technical reference material that supports the design, implementation, testing, deployment, and maintenance of RMA Tracker. These materials are intended for developers, testers, maintainers, and future contributors. They complement the preceding chapters by collecting diagrams, schemas, commands, reference tables, and engineering artifacts in a single location.

14.1. Package Diagrams

Provide UML package diagrams illustrating the relationships between the model, storage, service, UI, utility, and application packages. Each diagram should identify package responsibilities, dependencies, and permitted interactions.

14.2. Class Diagrams

Include UML class diagrams for all major classes such as MainWindow, RmaFormDialog, RmaRepository, DatabaseManager, BackupService, RmaSearchService, RmaRecord, RepairItem, ShippingInfo, and StatusHistory. Show inheritance, composition, aggregation, and associations.

14.3. Database Schema

Document the complete SQLite schema including CREATE TABLE statements, indexes, constraints, triggers (if added), and schema version information. Each table should include a description of its purpose.

14.4. Entity Relationship Diagram

Present a complete ERD showing relationships between RMA records, repair items, shipping information, customers, and status history records. Identify primary keys and foreign keys.

14.5. Sequence Diagrams

Provide sequence diagrams for Create RMA, Edit RMA, Search, Report Generation, Backup Creation, Restore Database, and Application Startup.

14.6. Activity Diagrams

Document workflows for creating an RMA, updating repair information, printing reports, exporting data, and restoring backups.

14.7. Deployment Diagram

Illustrate the runtime environment, including the desktop application, embedded Java runtime, SQLite database, local file system, reports, exports, and backup storage.

14.8. Project Directory Tree

Include the complete source tree with descriptions for `src/main/java`, `src/test/java`, resources, build scripts, documentation, and packaging assets.

14.9. Dependency Inventory

List every external dependency with version number, purpose, license, and update strategy. Include Java, Gradle, SQLite JDBC, PDF libraries, Excel libraries, and JUnit.

14.10. Build and Packaging Commands

Provide commonly used Gradle and `jpackage` commands for clean builds, testing, packaging, and release creation, along with explanations of each command.

14.11. Database Queries and Scripts

Store initialization SQL, maintenance queries, migration scripts, backup validation queries, and useful administrative scripts.

14.12. Test Case Catalog

Reference all manual and automated test cases. Include test IDs, objectives, prerequisites, execution steps, expected results, and pass/fail status.

14.13. Technical Glossary

Define technical terms, acronyms, architectural terminology, library names, and abbreviations used throughout Volume II.

14.14. References

List technical standards, Java documentation, Gradle documentation, SQLite documentation, IEEE software engineering standards, and other reference materials used during development.

14.15. Revision History

Maintain a chronological history of architectural changes, schema revisions, major features, bug fixes, documentation revisions, release dates, and approval records.